

Class templates:

List.h	
1	<code>#ifndef LIST_H</code>
2	<code>#define LIST_H</code>
3	
4	<code>template <class T></code>
5	<code>class List {</code>
6	<code>public:</code>
7	
8	
9	
10	
11	
12	
13	
14	
15	
16	
17	
18	<code>private:</code>
19	
20	<code>};</code>
21	
22	<code>#endif</code>

Two Basic Implementations:

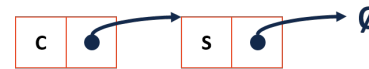
1.

2.

Linked Memory:



Snippet: ListNode struct	
1	<code>struct ListNode {</code>
2	<code> T data;</code>
3	<code> ListNode * next;</code>
4	<code> ListNode(T & data) : data(data), next(nullptr) { }</code>
5	<code>};</code>



Coding with Linked Lists: Examples

List.h	
1	#ifndef LIST_H
2	#define LIST_H
3	
4	template <class T>
5	class List {
6	public:
...	/* ... */
28	private:
29	struct ListNode {
30	T data;
31	ListNode * next;
32	ListNode(T & data) : data(data), next(nullptr) { }
33	};
34	
35	
36	
37	
38	
39	
40	};
41	
42	#include "List.cpp"
43	
44	#endif

List.cpp	
1	template <class T>
2	void List<T>::insertAtFront(const T& t) {
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	}

List.cpp	
1	template <class T>
2	void List<T>::printReverse() const {
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	}

List.cpp	
1	template <class T>
2	void List<T>::print_(ListNode *cur) const {
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	}

CS 225 – Things To Be Doing:

1. Lab 2 is due March 8
2. MP2 is due March 16
3. MP2 Extra Credit w/ Part 1 due March 9